From REINFORCE to GRPO

A worked tutorial, with the four things that actually bit us

assay — Phase 0.1 companion document

2026-08-02

Abstract

This document builds GRPO from REINFORCE one modification at a time, deriving each step rather than asserting it, and stops at every point where the derivation makes a promise that our own measurements broke. It is written to be read linearly. Sections 1 to 4 are the ladder: the objective, the score-function estimator, the baseline, and the group baseline that gives GRPO its name. Sections 5 and 6 add the two remaining pieces of full GRPO. Sections 7 and 8 map all of it onto the seven-run ladder and four ablations of Phase 0.1. Section 9 is the part worth the most: four places where the standard account is either incomplete or wrong at our scale, each one found by a measurement in this phase rather than taken from the literature.

Every numerical claim in Sections 8 and 9 comes from the clean ladder run of 2026-08-02 — ten arms, one commit, one GPU tier, 200 steps each — and regenerates from committed artefacts.

Contents

1 Setup: what problem are we actually solving?

1.1 The objects

Fix a pretrained language model with parameters θ. Write:

- x — a *prompt*, drawn from a task distribution $\mathcal{D}$. In Phase 0.1, x is a three-digit addition problem such as `"What is 471 + 638?"`.
- $y = (y_1, \ldots, y_T)$ — a *completion*, a sequence of T tokens.
- $\pi_\theta(y \mid x) = \prod_{t=1}^{T} \pi_\theta(y_t \mid x, y_{<t})$ — the policy. This is just the language model's own next-token distribution, applied autoregressively.
- $R(x, y) \in \mathbb{R}$ — the *reward*, computed by a grader after the completion is finished. In Phase 0.1, $R \in \{0, 1\}$: one if the final integer in y equals the correct answer, zero otherwise.

The objective is the expected reward:

$$J(\theta) \;=\; \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\big[R(x, y)\big]. \tag{1}$$

We want $\nabla_\theta J(\theta)$ so we can do gradient *ascent*.

1.2 Two rewards: proxy and true

There is a second reward, and the distinction between them is the instrument this entire project is built around. Every completion is graded *twice*:

- $R_{\text{proxy}}(x, y)$ — what the policy is **trained on**. It enters the loss. This is the grader you can actually run at training time.
- $R_{\text{true}}(x, y)$ — what you actually **want**. It is computed at every step and **never enters the loss**. Measured only.

The quantity of interest is their difference,

$$\text{gap} \;=\; R_{\text{proxy}} - R_{\text{true}}, \tag{2}$$

which is reward the proxy awards that the truth does not endorse — Goodhart's law, as a number.

In Phase 0.1, R_{true} is *always* R_{binary}; only the proxy varies:

Table 1. The three reward variants. Only the proxy changes, which is what makes gaps comparable across arms.

Variant	Used by	R_{proxy}	Gap
`binary`	ladder runs 1–7	R_{binary}	$\equiv 0$ by construction
`format`	ablation B	shape only; the answer is discarded	grows: measured +0.507
`tiebreak`	ablation C	$R_{\text{binary}} + 0.001\,\lvert y\rvert$	exactly $0.001\,\lvert y\rvert$: measured +0.042

The ladder's gap is identically zero, and that is correct. Runs 1–7 set proxy = true, so Equation (2) vanishes. The ladder is about the *estimator* — whether the gradient works — not about Goodhart. A nonzero gap on run 7 would indicate a bug, not a finding. The gap only becomes a meaningful quantity once a deliberately different proxy is introduced, which is what ablations B and C do.

Ablation C doubles as a calibration of the instrument. Its gap is analytically known — the tie-breaker pays exactly 0.001 per token, so the gap *must* equal $0.001\,\lvert y\rvert$. Measured over the final ten steps: gap = 0.04188 against $0.001 \times 41.88 = 0.04188$, a ratio of 1.0000. When a measurement has a closed form, check it.

Why the true reward must never be trained on. This is a contamination argument, not a cost one. The moment a held-out grader enters the objective it stops measuring generalisation and becomes a second proxy — and the gap it existed to reveal closes for the wrong reason. The asymmetry *is* the instrument.

The real-world version this rehearses: in the `bisect` environment, the proxy is "the failing test passes" and the truth is "a held-out suite exercising the same root cause through other code paths." You cannot train on the latter — if you could specify it perfectly, you would not need RL.

And a caveat that has to travel with every gap ever reported here: **held-out graders are graders too.** They can be wrong in both directions, so a gap is only as trustworthy as the true grader's own false-positive and false-negative rates on a hand-labelled gold set.

1.3 Why this is a bandit, not an MDP

Textbook RL sets up states, actions, transitions and discounted returns. Almost none of that machinery is needed here, and carrying it around obscures what is going on.

The reason: **reward arrives once, at the end, and the "environment" is deterministic.** The model emits a completion; a grader looks at it; a number comes back. There is no stochastic transition to model, no intermediate reward to discount, no value function that has to bootstrap across timesteps.

Formally this is a *contextual bandit*: context x, action y (a whole completion, chosen from an astronomically large action space), scalar payoff $R(x, y)$. Everything below is the bandit policy gradient. When you see A_i called an "advantage," remember it is not the $Q - V$ of temporal difference learning; it is just a centred reward.

> **Why this matters for reading the code.** `loop.py` assigns one scalar advantage per *rollout*, shared by all its tokens, and there is no per-timestep credit assignment anywhere. That is not a simplification — it is what the bandit formulation implies. Terminal reward and deterministic transitions mean every token in a completion is equally "responsible" for the outcome, as far as this estimator is concerned.

1.4 Vocabulary: rollout, episode, group, batch

These get used loosely in the literature. In this project they mean:

- **rollout** (= **episode**): one sampled completion y for one prompt x, with its reward. These are synonyms.
- **group**: the G rollouts sampled from the *same* prompt x. Phase 0.1 uses $G = 8$. The group is the unit that makes GRPO work, so it is never split.
- **batch**: all rollouts used for one gradient step. Phase 0.1 uses 16 prompts × 8 rollouts = 128 rollouts per step.

1.5 Notation for averages

Two different averages of reward appear throughout, over the two units just defined. They are *always* subscripted here, because conflating them is an easy and consequential mistake — it is the entire difference between rung 2 and rung 3 of the ladder.

$$\bar{r}_{\text{group}} = \frac{1}{G}\sum_{j=1}^{G} r_j \qquad \text{mean over the } G \text{ rollouts of } \textit{one} \text{ prompt,} \tag{3}$$

$$\bar{r}_{\text{batch}} = \frac{1}{N}\sum_{i=1}^{N} r_i \qquad \text{mean over all } N \text{ rollouts in a gradient step.} \tag{4}$$

Distinguish both from p, which denotes the *true* success probability $\Pr[R = 1]$ — a property of the policy and the task, not of any particular sample. We never observe p; we observe $\bar{r}$, which is an unbiased but noisy estimate of it. At $G = 8$, $\bar{r}_{\text{group}}$ takes only nine possible values, so it is a *very* noisy estimate of that prompt's difficulty. This matters more than it looks (Sections 3.3 and 9.3).

2 REINFORCE: the score-function estimator

2.1 The derivation

We want $\nabla_\theta J$. The difficulty is that θ appears inside the distribution we are averaging over, not inside the thing being averaged — R does not depend on θ at all. So we cannot simply push the gradient inside the expectation.

Fix x and write $J_x(\theta) = \sum_y \pi_\theta(y \mid x) R(x, y)$. Then

$$\nabla_\theta J_x(\theta) = \sum_y \nabla_\theta \pi_\theta(y \mid x)\, R(x, y) \tag{5}$$

$$= \sum_y \pi_\theta(y \mid x)\, \frac{\nabla_\theta \pi_\theta(y \mid x)}{\pi_\theta(y \mid x)}\, R(x, y) \tag{6}$$

$$= \sum_y \pi_\theta(y \mid x)\, \nabla_\theta \log \pi_\theta(y \mid x)\, R(x, y) \tag{7}$$

$$= \mathbb{E}_{y \sim \pi_\theta}\big[R(x, y)\, \nabla_\theta \log \pi_\theta(y \mid x)\big]. \tag{8}$$

Step (5) $\to$ (6) multiplies and divides by π_θ, which is legal wherever $\pi_\theta > 0$. Step (6) $\to$ (7) uses $\nabla_\theta \log f = \nabla_\theta f / f$ — the *log-derivative trick*, and the entire content of the derivation. It converts a gradient of a probability into an expectation, which is the one thing we can estimate by sampling.

Taking the outer expectation over prompts:

$$\boxed{\nabla_\theta J(\theta) \;=\; \mathbb{E}_{x,y}\big[R(x, y)\, \nabla_\theta \log \pi_\theta(y \mid x)\big].} \tag{9}$$

The quantity $\nabla_\theta \log \pi_\theta(y \mid x)$ is called the *score function*. Equation (9) is the policy gradient theorem for the bandit case.

2.2 The estimator

Equation (9) is an expectation, so replace it with a sample mean. Draw N rollouts (x_i, y_i), write $r_i = R(x_i, y_i)$ and $g_i = \nabla_\theta \log \pi_\theta(y_i \mid x_i)$. Then

$$\hat{g}_{\text{REINFORCE}} \;=\; \frac{1}{N} \sum_{i=1}^{N} r_i\, g_i \tag{10}$$

is an unbiased estimator of $\nabla_\theta J$. Ascend it. That is REINFORCE, complete.

In practice we do not compute g_i and then multiply — we build a scalar surrogate loss whose gradient is (10) and hand it to autograd:

$$\mathcal{L} \;=\; -\frac{1}{N} \sum_{i=1}^{N} r_i\, \log \pi_\theta(y_i \mid x_i), \qquad \nabla_\theta \mathcal{L} = -\hat{g}. \tag{11}$$

The minus sign turns ascent into descent. Note that r_i is a *constant* here: it is detached, never differentiated through. If you differentiate through the reward you are doing something else entirely.

2.3 The identity that everything later depends on

Claim 2.1 (The score has mean zero)**.** For any x, $\mathbb{E}_{y \sim \pi_\theta(\cdot \mid x)}\big[\nabla_\theta \log \pi_\theta(y \mid x)\big] = 0$.

Proof.

$$\mathbb{E}_y\big[\nabla_\theta \log \pi_\theta(y \mid x)\big] = \sum_y \pi_\theta(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x) = \sum_y \nabla_\theta \pi_\theta(y \mid x) = \nabla_\theta \sum_y \pi_\theta(y \mid x) = \nabla_\theta 1 = 0. \tag{12}$$

□

This is not a technicality. It is the load-bearing fact underneath baselines (Section 3), and Section 9 is largely the story of what happens when an innocuous implementation detail breaks it.

Read Claim 2.1 intuitively. $\nabla_\theta \log \pi_\theta(y \mid x)$ points in the direction that would make *this particular completion* more likely. Averaged over completions actually drawn from the policy, those directions cancel exactly — because the policy is already normalised, and making some outputs more likely necessarily makes others less so. There is no free "push everything up" direction. Keep this picture; Section 9.2 is about an implementation that quietly creates one.

3 The variance problem, and baselines

3.1 Why REINFORCE is unusable as written

$\hat{g}_{\text{REINFORCE}}$ is unbiased but has enormous variance. The cleanest way to see the problem: with $R \in \{0, 1\}$, Equation (10) is

$$\hat{g} = \frac{1}{N} \sum_{i\,:\,r_i=1} g_i, \tag{13}$$

a sum over the *correct* rollouts only. Failures contribute exactly nothing. Two consequences:

1. **Half the data is discarded.** At a 47% pass rate, 53% of the compute produces no gradient signal.
2. **Every surviving term has the same sign.** The estimator can only ever say "do more of this," never "do less of that." It is not contrastive.

3.2 The baseline, and why it is free

Subtract a constant b from every reward:

$$\hat{g}_b \;=\; \frac{1}{N} \sum_{i=1}^{N} (r_i - b)\, g_i. \tag{14}$$

Claim 3.1 (Baselines are unbiased)**.** If b does not depend on y, then $\mathbb{E}[\hat{g}_b] = \nabla_\theta J$ for every b.

Proof. $\mathbb{E}[(R - b)\nabla_\theta \log \pi_\theta(y \mid x)] = \mathbb{E}[R\nabla_\theta \log \pi_\theta(y \mid x)] - b\,\mathbb{E}[\nabla_\theta \log \pi_\theta(y \mid x)] = \nabla_\theta J - b \cdot 0 = \nabla_\theta J$, by Claim 2.1. □

So b is a free parameter: it changes the variance of the estimator without changing what the estimator estimates. This is the single most useful fact in policy-gradient methods.

The effect on the update is immediate and worth stating plainly. With $b > 0$, a failure ($r_i = 0$) now carries advantage $-b$, so its term is $-b \cdot g_i$ — pushing the policy *away* from that completion. **The estimator becomes contrastive.** Both consequences of Section 3 are addressed by one subtraction.

3.3 How much variance does it remove?

Take $R \in \{0, 1\}$ with $\Pr[R = 1] = p$, and define the conditional second moments of the score

$$E_c = \mathbb{E}\big[\|\nabla_\theta \log \pi_\theta(y \mid x)\|^2 \,\big|\, R = 1\big], \qquad E_w = \mathbb{E}\big[\|\nabla_\theta \log \pi_\theta(y \mid x)\|^2 \,\big|\, R = 0\big]. \tag{15}$$

Since all estimators here share the same mean (Claim 3.1), comparing variances is the same as comparing second moments. For constant b,

$$\mathbb{E}\big[(R-b)^2\|\nabla_\theta \log \pi_\theta(y \mid x)\|^2\big] \;=\; p(1-b)^2 E_c \;+\; (1-p)\, b^2 E_w. \tag{16}$$

Minimising over b gives the optimal baseline

$$b^\star \;=\; \frac{p\, E_c}{p\, E_c + (1-p) E_w}. \tag{17}$$

Equation (17) is not the thing anyone actually computes. Look at what $b^\star$ requires: p, E_c and E_w — the true success probability and two conditional second moments of the score. None of the three is observable. So every implementation substitutes the one quantity that is free, the sample mean reward $\bar{r}$ (Section 1.5), which is a plug-in estimate of p.

That substitution is exact only when $E_c = E_w$: setting them equal in Equation (17) collapses it to $b^\star = p$, estimated by $\bar{r}$. Otherwise using $\bar{r}$ is a heuristic — a *reasonable* one, since the correction factor is bounded and the resulting estimator is still unbiased by Claim 3.1, but a heuristic nonetheless. Section 9.3 is what our own measurements say when that assumption fails, and here it fails badly enough to erase the entire benefit.

Setting $E_c = E_w$ and comparing $b = 0$ against $b = p$:

$$b = 0: \quad p\, E, \tag{18}$$

$$b = p: \quad p(1-p)^2 E + (1-p)p^2 E \;=\; E\, p(1-p)\big[(1-p)+p\big] \;=\; E\, p(1-p), \tag{19}$$

so the variance ratio is

$$\boxed{\frac{\mathrm{Var}[\hat{g}_0]}{\mathrm{Var}[\hat{g}_p]} \;=\; \frac{pE}{Ep(1-p)} \;=\; \frac{1}{1-p}.} \tag{20}$$

At $p = 0.5$ the baseline halves the variance; at $p = 0.9$ it cuts it by a factor of ten. This is the pre-registered point prediction for ablation A (Section 8.1).

4 From a global baseline to a group baseline

4.1 The idea GRPO is named after

Claim 3.1 allows b to depend on the *prompt* x, since the expectation in Claim 2.1 is taken at fixed x. So we may use $b(x)$, and the variance-minimising choice is approximately

$$b(x) \;\approx\; \mathbb{E}\big[R \mid x\big], \tag{21}$$

the expected reward *on this prompt*. This is strictly better than a global b: an easy prompt and a hard prompt have very different expected rewards, and a single global constant is systematically wrong for both.

Classical actor–critic methods estimate $\mathbb{E}[R \mid x]$ with a learned value network $V_\phi(x)$ — a second model, with its own parameters, its own optimiser, and its own failure modes.

GRPO's contribution is to replace that network with a Monte Carlo estimate. Sample G completions for the same prompt and average their rewards:

$$\hat{b}(x) \;=\; \frac{1}{G}\sum_{j=1}^{G} r_j. \tag{22}$$

That is the whole idea. *Group Relative Policy Optimisation* is REINFORCE with a per-prompt baseline bought with extra samples instead of a critic network.

> **The trade being made.** A critic amortises: train it once, query it cheaply forever. A group baseline pays $G\times$ the sampling cost at every step, and buys in return: no second model, no critic-training instability, no value-function bias, and an estimate that is automatically correct for the current policy. At $G = 8$ you spend $8\times$ the rollouts to delete an entire neural network from the system. For LLM post-training, where sampling is the well-understood part and critic training is not, that is usually a good trade.

4.2 Leave-one-out

Equation (22) has a subtle flaw: r_i appears in its own baseline, so $\hat{b}$ is correlated with r_i and Claim 3.1 no longer applies exactly. The fix is to exclude the sample being centred:

$$A_i^{\text{loo}} \;=\; r_i - \frac{1}{G-1}\sum_{j\neq i} r_j. \tag{23}$$

Claim 4.1 (LOO is a uniform rescale of the mean-centred advantage)**.** With $S = \sum_j r_j$ and $A_i^{\text{mean}} = r_i - S/G$,

$$A_i^{\text{loo}} \;=\; \frac{G\,r_i - S}{G-1} \;=\; \frac{G}{G-1}\,A_i^{\text{mean}}. \tag{24}$$

Proof. $A_i^{\text{loo}} = r_i - \frac{S-r_i}{G-1} = \frac{(G-1)r_i - S + r_i}{G-1} = \frac{Gr_i - S}{G-1}$, and $A_i^{\text{mean}} = \frac{Gr_i - S}{G}$. □

At $G = 8$ the factor is $8/7 \approx 1.143$. So LOO differs from mean-centring by a constant scale — which is absorbed entirely by the learning rate, and *exactly* cancelled if you normalise by the standard deviation (Section 5). The bias argument for LOO is real but small; do not expect it to show up as a visible effect.

4.3 Worked example: what conditioning actually buys

This is the table that makes the difference concrete. Take $G = 8$, a batch whose overall pass rate is $\bar{r}_{\text{batch}} = 0.43$, and three groups of differing difficulty.

Table 2. The same rewards, three ways of centring them. Only the *group* baseline reacts to how hard the prompt is.

Group	Baseline	Advantages A_i
Easy: $r = (1,1,1,1,1,1,1,0)$	`none`	$+1,\ +1,\ +1,\ +1,\ +1,\ +1,\ +1,\ \ 0$
	`global`	$+0.57 \times 7,\ \ -0.43$
	`group_loo`	$+0.143 \times 7,\ \ \mathbf{-1.000}$
Hard: $r = (0,0,0,0,0,0,0,1)$	`none`	$0 \times 7,\ \ +1$
	`global`	$-0.43 \times 7,\ \ +0.57$
	`group_loo`	$-0.143 \times 7,\ \ \mathbf{+1.000}$
Average: $r = (1,0,1,1,0,0,1,0)$	`none`	$+1,\ 0,\ +1,\ +1,\ 0,\ 0,\ +1,\ 0$
	`global`	$+0.570/-0.430$
	`group_loo`	$+0.571/-0.571$

Read the three blocks in order:

- **Easy prompt.** Under `none`, seven rollouts on a problem the model *already solves* each receive full weight $+1$: the update is dominated by reinforcing existing competence. Under `group_loo` they get $+0.143$, while the single failure gets -1.000 — the strongest signal in the group. The group baseline has correctly identified that the only informative event here is the failure.
- **Hard prompt.** Under `none` the seven failures receive advantage exactly 0: a failure carries no information at all. Under `group_loo` the lone success gets the maximum $+1.000$.
- **Average prompt.** `global` and `group_loo` nearly coincide (0.570 vs 0.571) — because this group's pass rate equals the batch's.

The one-sentence summary of Section 4. The group baseline differs from the global one *only* when a prompt is harder or easier than average — and that difference *is* the conditioning. If every prompt in your task set had identical difficulty, rung 2 and rung 3 would be the same algorithm.

5 Advantage normalisation

Published GRPO divides the centred advantage by the group's standard deviation:

$$\tilde{A}_i \;=\; \frac{A_i}{\sigma_{\text{group}} + \varepsilon}, \qquad \sigma^2_{\text{group}} = \frac{1}{G}\sum_j \big(r_j - \bar{r}_{\text{group}}\big)^2. \tag{25}$$

Combined with mean-centring, this makes $\tilde{A}$ a *z-score* within the group.

Two properties follow, and both matter operationally.

Claim 5.1 (Bounded)**.** For any group of G values, $\max_i |\tilde{A}_i| \le \sqrt{G-1}$.

Proof. The z-scores satisfy $\sum_i \tilde{A}_i = 0$ and $\sum_i \tilde{A}_i^2 = G$. Fix $i = 1$ and write $z = \tilde{A}_1$. Then $\sum_{i\ge 2} \tilde{A}_i = -z$ and $\sum_{i\ge 2} \tilde{A}_i^2 = G - z^2$. Cauchy–Schwarz gives $\big(\sum_{i\ge 2} \tilde{A}_i\big)^2 \le (G-1)\sum_{i\ge 2} \tilde{A}_i^2$, i.e. $z^2 \le (G-1)(G-z^2)$, hence $z^2 G \le G(G-1)$ and $|z| \le \sqrt{G-1}$. □

At $G = 8$ this is $\sqrt{7} \approx 2.6458$, attained exactly by a lone winner (or lone loser). Phase 0.1 logs `max_abs_advantage` every step purely as a rig check: if it ever exceeds 2.6458, the advantage function is not computing a z-score and nothing downstream means anything.

Claim 5.2 (Scale-invariant)**.** $\tilde{A}$ is unchanged by any affine transform $r_i \mapsto \alpha r_i + \beta$, $\alpha \neq 0$.

This one has a sharp consequence that we got wrong on the first pass and had to correct before running anything. Claim 5.2 says the rewards $\{0, \ldots, 0, 1\}$ and $\{0.5, \ldots, 0.5, 0.500001\}$ produce *identical* normalised advantages. Normalisation keeps only the *shape* of reward differences and discards their *magnitude* entirely. We return to this in Section 8.3, because it is the whole content of ablation C.

5.1 Dr. GRPO's objection

Two conventions are contested in the literature. Published GRPO (i) divides by σ_{group} and (ii) length-normalises the log-probability sum. Dr. GRPO argues both introduce bias and removes them. Phase 0.1 keeps both as switches — `normalize_by_std` and `length_normalize` — precisely so the question is answered by measurement rather than by citation. Section 9.2 reports what we found, and it is not a small effect.

6 The KL leash

Nothing so far stops the policy from drifting arbitrarily far from where it started in pursuit of reward. The standard remedy adds a penalty against a frozen reference policy π_{ref} (the model as it was before RL):

$$\mathcal{L} = \underbrace{-\frac{1}{N}\sum_i A_i \log \pi_\theta(y_i \mid x_i)}_{\text{policy gradient}} + \beta \underbrace{\mathrm{KL}\left[\pi_\theta \,\|\, \pi_{\text{ref}}\right]}_{\text{leash}}. \tag{26}$$

6.1 Estimating the KL

We cannot compute KL in closed form over the space of completions, so we estimate it from the same samples. The estimator used here is Schulman's *k3*:

$$\hat{k}_3 = \rho - \log \rho - 1, \qquad \rho = \frac{\pi_{\text{ref}}(y \mid x)}{\pi_\theta(y \mid x)}. \tag{27}$$

It is unbiased — $\mathbb{E}_{\pi_\theta}[\rho] = 1$ and $\mathbb{E}_{\pi_\theta}[\log \rho] = -\mathrm{KL}[\pi_\theta \| \pi_{\text{ref}}]$, so $\mathbb{E}[\hat{k}_3] = 1 - (-\mathrm{KL}) - 1 = \mathrm{KL}$ — and, unlike the naive $-\log \rho$, it is non-negative pointwise, which makes it far better behaved as a penalty.

Apply k3 per token, not per sequence. Sequence-level probabilities are products over hundreds of tokens, so ρ compounds multiplicatively and its variance explodes with length. Phase 0.1 computes (27) at each token position and averages. This was a deliberate change during implementation, not an accident of the code.

6.2 Two implementation notes

Add it to the loss, not the reward. If the KL penalty is folded into R before advantages are computed, it passes through the group baseline — and a penalty that is roughly constant within a group is *subtracted away* by that baseline. The leash silently goes slack. It must enter at (26), after centring.

Under LoRA, the reference is free. With a frozen base model and only LoRA adapters trainable, π_{ref} is

exactly the base model — obtained by disabling the adapter. No second set of weights is needed, only a second forward pass.

7 The ladder

Everything above is now one dataclass. Each rung differs from the previous one by exactly one field, which is what makes any observed difference attributable.

Table 3. The Phase 0.1 ladder. Runs 4–6 are cut: under the pinned single-epoch design the importance ratio is identically 1, so clipping is a no-op, and the remaining two switches are isolated by ablations B and C instead.

Run	Name	Configuration	What it adds
1	REINFORCE	`baseline="none"`	—
2	+ mean baseline	`baseline="global"`	contrastive updates
3	+ group baseline	`baseline="group_loo"`	conditioning on the prompt
7	full GRPO	+ `normalize_by_std`, `kl_coef`	scale-free advantages, a leash

The rung $1 \to 2 \to 3$ progression is worth stating carefully, because it is easy to conflate the last two steps:

- $1 \to 2$ introduces **centring**. Failures acquire negative weight; the update becomes contrastive.
- $2 \to 3$ introduces **conditioning**. The centre becomes prompt-specific, so an easy prompt and a hard prompt are judged against their own difficulty rather than the batch's average. Table 2 is exactly this difference.

8 The four ablations

Phase 0.1's deliverable is four *deliberate breakages*, each with a pre-registered signature recorded before any training run, and each carrying two distinct failure branches: *prediction wrong* (a finding, to be reported) and *rig broken* (a bug, to be fixed). All numbers below come from the clean ladder of 2026-08-02, one commit, one GPU tier, seed 0.

8.1 Ablation A — remove the baseline

Isolates: the baseline itself, rung 1 versus rung 2. **Prediction:** Equation (20) — the no-baseline estimator should have $1/(1-p)$ times the noise-to-signal ratio of the baselined one.

This ablation is the subject of Sections 9.2 to 9.4, because both the original measurement and the theory behind it turned out to be wrong. It is the most instructive of the four.

8.2 Ablation B — remove the KL leash on a degenerate reward

Isolates: β. Because B needs *two* changes from run 7 (no leash *and* a degenerate reward), it is measured against a control that has the degenerate reward but keeps the leash; the difference between B and its control isolates β alone.

What "degenerate" and "hackable" mean

Two terms that get used loosely, and which are worth separating precisely because they are properties of *different things*.

A grader is **degenerate** when its score does not depend on whether the task was performed. For R_{format} this is not a figure of speech — it is one line of the implementation:

```
def grade_format_only(completion, expected):
  del expected   # deliberately unused — that *is* the pathology
  ...  return reward 1.0 if completion matches <answer>N</answer> else 0.0
```

The correct answer is *discarded before grading*. Anything shaped like `<answer>N</answer>` pays 1.0, so the constant string `<answer>0</answer>` scores full marks on every prompt in the distribution. **The reward-maximising policy is a constant function that never reads its input.**

"Degenerate" is meant in the mathematical sense — a structure has collapsed. A degenerate conic is a pair of lines rather than an ellipse; a degenerate reward is one whose level sets no longer separate correct from incorrect.

A grader is **hackable** when a degenerate maximum both exists *and* is cheaply reachable from where the policy starts. The distinction matters:

- **Degeneracy** is a property of the grader alone — inspectable statically, without running anything.
- **Hackability** is a property of the (grader, policy, budget) triple. A degenerate grader whose exploit is astronomically unlikely under the base policy will not be hacked in 200 steps, because RL at this scale *amplifies what is already in the policy's support* rather than discovering new behaviour.

That second point is why this project screens $p_{\text{hack}}@64$ on the *base* policy before admitting any grader variant, and why "nothing hacks at 1.7B in 200 steps" is recorded as its single largest risk. A diagnostic that flags degeneracy is measuring the grader; whether that degeneracy *bites* depends on the model and the budget.

"Deliberately" hackable means R_{format} was built this way as a *positive control*. If one claims a diagnostic detects grader degeneracy, at least one environment must have a known answer — otherwise a null result cannot be interpreted, since "the diagnostic failed" and "there was nothing to find" look identical.

What it looks like when the grader is hacked

Table 4. Ablation B's control arm over training. The proxy is learned in ten steps; the skill never is.

step	proxy	true	gap	distinct completions
0	0.102	0.516	−0.414	97
5	0.453	0.352	+0.102	97
10	0.961	0.484	+0.477	51
20	0.992	0.297	+**0.695**	66
199	0.984	0.445	+0.539	80

Three things to read off Table 4.

The gap starts negative. The base model already solves 51.6% of the additions but emits the required tag only 10.2% of the time, so initially the proxy *understates* true ability. Reward hacking is not visible at step 0; it has to be trained in.

The proxy saturates in ten steps and the skill never moves. True reward wanders — $0.516 \to 0.297 \to 0.445$ — and *ends below where it started*. Optimising the format actively damaged arithmetic before partially recovering it.

The completions make it unambiguous. On `875 + 812` (answer: 1687), the policy at step 150 emitted `<answer>1677</answer>`, `<answer>1707</answer>`, `<answer>1677</answer>`, `<answer>1756</answer>` — perfectly formatted, every one wrong. Two hundred steps of gradient descent spent learning punctuation.

This is why the true reward must never be trained on. R_{binary} is computed at every step of ablation B and thrown away — it never touches the loss. The moment a held-out grader enters the objective it stops measuring generalisation and becomes a second proxy, and the gap it was supposed to reveal closes for the wrong reason. The asymmetry is the instrument.

Table 5. Ablation B. Proxy is what is optimised; true is R_{binary}, measured and never trained on. The gap is the Goodhart quantity.

Arm	proxy	true	gap	KL end	KL share of loss
control ($\beta = 0.04$)	0.993	0.486	+0.507	2.68	0.490
ablation B ($\beta = 0$)	0.999	0.510	+0.489	2.45	0.000

Two things to take from Table 5. First, the degenerate grader is *comprehensively* hacked: the policy scores 0.993 on the proxy while true performance sits at 0.486 — it learned the format and not the task. That +0.507 gap is the largest in the whole ladder. Second, and against the pre-registered prediction, **removing the leash changes the gap by** −0.018**, i.e. nothing** — despite the KL term carrying 49% of the loss in the control arm.

The mechanism turns out to run through ablation D: the degenerate reward drives the dead-group fraction to 0.888, so there is almost no policy gradient left for the leash to restrain. A leash is irrelevant when the horse has stopped.

8.3 Ablation C — add a tiny tie-breaker

Isolates: the reward's magnitude structure. Replace R_{binary} with

$$R_{\text{tiebreak}}(x, y) \;=\; R_{\text{binary}}(x, y) \;+\; 0.001 \cdot |y|. \tag{28}$$

The pre-registered signature for this ablation was originally "divide-by-≈ 0 advantage spikes." Claims 5.1 and 5.2 say that is **unreachable**: the normalised advantage is a z-score, bounded by $\sqrt{G-1}$ and invariant to scale. We rewrote the signature before running anything.

What actually goes wrong is sharper, and is this project's thesis in miniature. Because z-scoring discards magnitude, a group that was unanimous under the binary reward — and therefore contributed *zero* gradient — becomes, on adding a 0.001 tie-breaker, **fully alive at full gradient magnitude**, with the advantages ordered entirely by completion length.

Table 6. Ablation C against its parent. Goodhart emerging from the optimiser's own arithmetic.

Arm	true reward	gap	dead groups	tokens/rollout
run 7 (binary)	0.592 → 0.907	+0.000	0.468	17.2
ablation C (tiebreak)	0.598 → 0.728	+0.042	**0.008**	**35.5**

All four predicted signatures fire: dead groups collapse 0.468 → 0.008, completions double, the true-reward

gain falls from +0.315 to +0.130, and a real proxy–true gap of +0.042 opens on a task where run 7's gap is identically zero.

8.4 Ablation D — force every group unanimous

Isolates: the dead-group failure mode, by construction.

Claim 8.1 (Dead-group probability)**.** If a prompt has per-sample success probability p, the probability that a group of G i.i.d. rollouts is unanimous — and therefore produces *zero* gradient under a group baseline — is

$$P_{\text{dead}}(p) \;=\; p^G + (1-p)^G. \tag{29}$$

This function is convex, symmetric about $p = 1/2$, and minimised there. At $G = 8$ it is 0.008 at $p = 0.5$ but 0.66 at $p = 0.05$ *or* $p = 0.95$. Two consequences that are easy to miss:

- **It is blind to sign.** A group that is unanimously correct (saturated) and one that is unanimously wrong (unreachable) are both dead, and behave oppositely under training — one will stay dead, one may become live. The scalar cannot tell you which.
- **The mean hides it.** By Jensen's inequality, $\mathbb{E}[P_{\text{dead}}(p)] \geq P_{\text{dead}}(\mathbb{E}[p])$ for convex P_{dead}. A task set that is half trivial and half impossible has mean pass rate 0.5 and *every* group dead; a task set genuinely centred at 0.5 has 0.8% dead. Identical means, a 55× difference in wasted compute. This is why Phase 0.1's task was selected on the dead-group fraction rather than on pass rate.

Measured: with every group forced unanimous, `frac_degenerate_groups` = 1.000 on all 200 steps, `grad_norm` = 0.0000 on all 200 steps, true reward flat at 0.503 → 0.509, and 313,312 tokens generated at the parent's rate. **Thirty-eight minutes of GPU for exactly zero gradient.**

8.5 C and D are one event with two faces

The two ablations are the same phenomenon, separated by whether the reward has a tie-breaking term:

Table 7. What a unanimous group does, as a function of reward structure.

Unanimous group, reward is. . .	**Result**
clean binary	zero gradient — the step is wasted (D)
carrying any tie-breaker	full-magnitude gradient chasing noise (C)

9 Four things the standard account gets wrong at this scale

This section is the reason the document exists. Each subsection is a place where the clean derivation above makes a promise that our own measurements broke.

9.1 Dead groups are a pathology GRPO *introduces*

A group is dead when all its advantages are zero. Under which baselines does that happen?

- `none`: $A_i = r_i$, so the group is dead only if *every* rollout failed.
- `global`: $A_i = r_i - \bar{r}_{\text{batch}}$. Since $\bar{r}_{\text{batch}}$ is an average over 128 rollouts and r_i is binary, A_i is essentially *never* exactly zero. Dead groups cannot occur.
- `group_loo`: $A_i = 0$ for all i exactly when the group is unanimous, either way.

Measured on the clean ladder:

Table 8. Dead-group fraction by baseline, averaged over 200 steps. The analytical prediction, exactly.

Baseline	Dead when…	Measured
`none`	the group is all-wrong	0.037
`global`	never	**0.000**
`group_loo`	the group is unanimous, either way	**0.454**
`forced` (D)	always, by construction	1.000

Read the middle two rows against each other. The same unanimous group that GRPO throws away is perfectly alive under a global baseline, where it contributes $\pm b$. Conditioning the baseline on the prompt — the thing that makes GRPO better — is *also* what creates the dead-group pathology.

And it *worsens as training succeeds*: under `group_loo` the dead fraction climbs from 0.244 over the first ten steps to 0.606 over the last ten, because the policy is solving more prompts unanimously. Full GRPO (run 7) is worse still, 0.300 → 0.694. The better the policy gets, the larger the share of each batch that produces no gradient at all — a built-in brake that tightens exactly as the task is being learned.

This is not a curiosity: `prime-rl`, Prime Intellect's production RL stack, ships a `zero_advantage` pre-batch filter *on by default* to drop exactly these rollouts. The pathology is recognised industry-wide. What we add is measuring its size per configuration.

9.2 Length normalisation breaks Claim 2.1

Implementations divide each rollout's summed log-probability by its token count, so that a 40-token rollout does not contribute four times the gradient of a 10-token one:

$$\mathcal{L} = -\frac{1}{N}\sum_i A_i \cdot \frac{1}{|y_i|}\log \pi_\theta(y_i \mid x_i). \tag{30}$$

This looks innocuous. It is not. Claim 2.1 relied on the sum telescoping, $\sum_y \nabla_\theta \pi_\theta = \nabla_\theta \sum_y \pi_\theta = \nabla_\theta 1 = 0$. With a per-sample weight $w(y) = 1/|y|$ inside the sum, that telescoping fails:

$$\mathbb{E}_y\left[\tfrac{1}{|y|}\nabla_\theta \log \pi_\theta(y \mid x)\right] \;=\; \sum_y \pi_\theta(y \mid x)\,\tfrac{1}{|y|}\,\nabla_\theta \log \pi_\theta(y \mid x) \;=\; \sum_y \tfrac{1}{|y|}\nabla_\theta \pi_\theta(y \mid x) \;\neq\; \nabla_\theta \sum_y \pi_\theta(y \mid x) \;=\; 0, \tag{31}$$

because $1/|y|$ cannot be pulled out of the sum. **The score no longer has mean zero**, so Claim 3.1 fails and a baseline is no longer free: it changes the estimand.

We measured this directly with a paired fixed-policy probe — one policy state, 40 batches, the *same* rollouts scored under all three baselines, so nothing but the baseline differs. The statistic is the cosine between the three arms' mean gradients: if all three estimate the same thing, these should be ≈ 1.

Table 9. Do the three baselines estimate the same gradient? Mean over 3 seeds, $N = 40$ batches.

	`none` | `global`	`none` | `group_loo`	`global` | `group_loo`
`length_normalize=True`	+0.727	+0.734	+0.996
`length_normalize=False`	**+0.991**	**+0.992**	+0.999

With length normalisation on, the two *centred* estimators agree with each other almost exactly (0.996) while the uncentred one sits roughly 40° away from both. Turn length normalisation off and all three snap into agreement. Claim 2.1 is restored, and with it the premise that makes ablation A a well-posed question at all.

This has teeth beyond ablation A. The ladder's primary arms all run with `length_normalize=True`. In that configuration the rungs are not estimating a common quantity, so "does the baseline reduce variance" has no well-defined answer — *that* is the finding, not a failure to measure. Separately, removing length normalisation improved the noise-to-signal ratio of *every* arm (0.45–0.94 → 0.30–0.33) and improved final true reward on both arms tested (0.907 → 0.927 and 0.728 → 0.844), with the proxy–true gap unchanged. Two independent lines of support for Dr. GRPO's position, arrived at by measurement.

9.3 The variance reduction does not materialise

With the estimators made comparable (`length_normalize=False`), we can finally ask ablation A's actual question. Define the scale-free noise-to-signal ratio for baseline b,

$$\mathrm{NSR}_b \;=\; \frac{V_b}{\|\bar{g}_b\|^2}, \qquad V_b = \frac{1}{N}\sum_{n=1}^{N}\left\|g_n^{(b)} - \bar{g}_b\right\|^2, \tag{32}$$

where $g_n^{(b)}$ is the full-batch gradient of batch n under baseline b. Scale-freeness is required, not cosmetic: the arms produce gradients of very different magnitudes, so comparing raw variance would report that scale difference instead.

Equation (20) predicts $\mathrm{NSR}_{\texttt{none}}/\mathrm{NSR}_{\texttt{global}} = 1/(1-p)$. At the measured $p = 0.466$ that is 1.87.

Table 10. Observed ratio against the point prediction, $N = 40$ batches, paired bootstrap.

Seed	ratio	95% CI	predicted $1/(1-p)$
0	0.859	[0.616, 1.219]	1.869
1	0.921	[0.704, 1.155]	1.825
2	1.350	[0.969, 1.873]	1.922

Every interval contains 1.0 — no reduction is demonstrated — and every interval *excludes* the prediction. Mean ratio 1.04. **At $p \approx 0.47$ on this task, the baseline delivers no detectable variance reduction, and the textbook magnitude is ruled out.**

Why? Recall from Section 3.3 that Equation (20) assumed $E_c = E_w$. Solve Equation (16) for the ratio at which the benefit vanishes entirely:

$$\begin{aligned} pE_c &= p(1-p)^2E_c + (1-p)p^2E_w \\ E_c\left[1-(1-p)^2\right] &= (1-p)\,p\,E_w \\ E_c\,p(2-p) &= (1-p)\,p\,E_w \end{aligned}$$

$$\boxed{\frac{E_w}{E_c} = \frac{2-p}{1-p} \;=\; 2.87 \;\text{ at } p = 0.466.} \tag{33}$$

So if incorrect completions carry roughly 2.9× the squared score norm of correct ones, the baseline's benefit

disappears exactly. That is entirely plausible — an incorrect completion is one the policy assigned lower probability to, and $\|\nabla_\theta \log \pi_\theta\|$ is larger there. Equation (33) is a *derived, falsifiable number*, and measuring E_w/E_c directly is the obvious next probe.

The methodological point, which outlives the result. On the first pass, with length normalisation on, the observed ratio was 1.786 against a prediction of 1.872 — a 4.6% match, and we nearly reported it as a clean confirmation. Decomposing NSR showed the agreement was coincidental: theory predicts *signal unchanged, noise down* 1.87×, whereas what actually happened was *noise up* 2.34×, *signal up* 4.16×. Two effects the theory does not contain, pulling in opposite directions, landing on the right number. The rig check — "do these three estimators even share a mean?" — is the only thing that caught it.

9.4 The half-batch cosine cannot compare baseline arms

Ablation A was originally measured by splitting each batch in half, computing both half-gradients, and taking their cosine. Two independent samples of the same expected gradient should agree; the cosine is then a direct read on estimator noise, with no trend confound.

It came back *reversed*: $\rho_2/\rho_1 = 0.154$ against a pre-registered gate of ≥ 2.0, with all rig checks clean. The reason is that both arms are confounded, in opposite directions, and neither confound is about variance.

- `baseline="none"` — every advantage is ≥ 0, so *both* halves weight the shared component of $\nabla_\theta \log \pi_\theta(y \mid x)$ positively. The cosine is **inflated** by precisely the nuisance direction a baseline exists to remove.
- `baseline="global"` — the baseline is the *full-batch* mean while the cosine splits that same batch. Writing $b = (b_A + b_B)/2$, where b_A, b_B are the two *half*-batch mean rewards, each half carries a term $\pm(b_A - b_B)/2 \cdot \sum \nabla_\theta \log \pi_\theta(y \mid x)$: an anti-correlated component **manufactured by the split boundary**. The cosine is **deflated**.
- `baseline="group_*"` — advantages sum to zero *within* each group and groups are never split, so each half sums to zero independently. No coupling. This arm was always clean.

The obvious repair — centre each half on its own mean advantage before measuring — does not work, and the reason is worth internalising. Under `none` the centred advantage is $r_i - \bar{r}_A$ (writing $\bar{r}_A$ for the mean reward over half A); under `global` it is $(r_i - b) - (\bar{r}_A - b) = r_i - \bar{r}_A$. The constant cancels: **centring makes the two arms the same estimator**, erasing the contrast instead of the confound. Verified numerically — at a shared policy state the two agree to six decimal places.

That is why ablation A needs the fixed-policy probe of Section 9.3 rather than a comparison between training arms. Two training arms follow different trajectories, so any difference between them confounds the estimator with the policy state it was measured at. Only a probe that holds the policy fixed and varies *nothing but the baseline* can answer the question.

The general lesson. The half-batch cosine measures directional *reproducibility of the applied update*. A degenerate estimator that always points the same way scores near 1.0 while learning nothing from reward. So a metric that rewards a large common-mode component will always report that removing it hurt — which is Goodhart's law operating inside the diagnostic itself. For a project whose entire subject is the gap between a target and the proxy used to measure it, finding that gap in our own instrument is not an embarrassment. It is the thesis.

10 Summary

1. **REINFORCE** is the log-derivative trick plus a sample mean: $\hat{g} = \frac{1}{N} \sum r_i g_i$. Unbiased, unusably noisy, and non-contrastive when $R \geq 0$.
2. **A baseline** is free because $\mathbb{E}[\nabla_\theta \log \pi_\theta(y \mid x)] = 0$. It makes the update contrastive and, in theory, cuts variance by $1/(1 - p)$.
3. **GRPO's group baseline** is a Monte Carlo estimate of $\mathbb{E}[R \mid x]$ bought with G rollouts instead of a critic network. It differs from a global baseline exactly when prompts differ in difficulty.
4. **Normalisation** makes advantages z-scores: bounded by $\sqrt{G - 1}$ and scale-invariant. The scale-invariance is what makes ablation C possible.
5. **The KL leash** uses the per-token k3 estimator and must be added to the loss, not the reward, or the group baseline subtracts it away.

And the four things measurement changed:

1. Dead groups are a pathology the *group* baseline introduces — 0.000 under a global baseline, 0.454 under GRPO's.
2. Length normalisation breaks $\mathbb{E}[\nabla_\theta \log \pi_\theta(y \mid x)] = 0$, so under the ladder's primary configuration the rungs do not estimate a common quantity at all.
3. Once they do, the predicted variance reduction **does not appear** — consistent with $E_w/E_c \approx 2.9$, a derived and testable number.
4. The half-batch cosine cannot compare baseline arms, and the obvious fix erases the contrast rather than the confound.